УНИВЕРСИТЕТ ИТМО

Факультет ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА № 2

по дисциплине

«Проектирование вычислительных систем»

Тема: «Последовательный интерфейс UART»
Вариант № 5 («Кодовый замок»)

Выполнили:
Студенты группы P3432
Носов Георгий Иванович
Карандашева Анастасия Денисовна

Преподаватель:
Ключев Владислав Аркадьевич

г. Санкт-Петербург
2026

Содержание

Содержание 2

1. Задание 3

Общие требования к драйверу UART 3

Спецификация варианта № 5 («Кодовый замок») 3

2. Архитектурное описание программы и драйверов UART 4

3. Аппаратные особенности интерфейса UART на стенде SDK-1.1M 5

4. Реализация драйвера UART в режимах опроса и прерываний 5

Режим циклического опроса флагов (Polling, без прерываний) 6

Режим прерываний с кольцевой буферизацией FIFO 6

5. Конечный автомат кодового замка 6

Результаты практического тестирования программы на стенде 7

6. Исходные коды программы собственной разработки 9

Файл Core/Inc/ring_buffer.h (Кольцевой буфер FIFO) 9

Файл Core/Inc/uart_driver.h (Заголовочный файл драйвера UART) 10

Файл Core/Src/uart_driver.c (Реализация драйвера UART) 10

Файл Core/Src/lock_fsm.c (Реализация конечного автомата замка) 12

Файл Core/Src/main.c (Фрагмент главного суперцикла программы) 15

Файл Core/Src/stm32f4xx_it.c (Обработчик прерывания USART6_IRQHandler) 16

7. Выводы 16

1. Задание

Цели работы:

1. Изучить протокол передачи данных по интерфейсу UART.

2. Получить базовые знания об организации системы прерываний в микроконтроллерах на примере микроконтроллера STM32.

3. Изучить устройство и принципы работы контроллера интерфейса UART, получить навыки организации обмена данными по UART в режимах опроса и прерываний.

Общие требования к драйверу UART

Разработать и реализовать два варианта драйверов UART для стенда SDK-1.1M: с использованием и без использования прерываний. Драйверы, использующие прерывания, должны обеспечивать работу в «неблокирующем» режиме (возврат из функции происходит сразу же, без ожидания окончания приема/отправки), а также буферизацию данных для исключения случайной потери данных. В драйвере, не использующем прерывания, функция приема данных также должна быть «неблокирующей», то есть она не должна зависать до приема данных (которые могут никогда не поступить). При использовании режима «без прерываний» прерывания от соответствующего блока UART должны быть запрещены.

Написать с использованием разработанных драйверов программу, которая выполняет определенную вариантом задачу. Для всех вариантов должно быть реализовано два режима работы программы: с использованием и без использования прерываний. Каждый принимаемый стендом символ должен отсылаться обратно, чтобы он был выведен в консоли (так называемое «эхо»). Каждое новое сообщение от стенда должно выводиться с новой строки. Если вариант предусматривает работу с командами, то на каждую команду должен выводиться ответ, определенный в задании или «ОК», если ответ не требуется. Если введена команда, которая не поддерживается, должно быть выведено сообщение об этом.

Спецификация варианта № 5 («Кодовый замок»)

Доработать программу кодового замка. Теперь ввод кода должен происходить не с помощью кнопки стенда, а по UART. После ввода единственно верной последовательности из восьми латинских букв без учёта регистра и цифр должен загореться зелёный светодиод, обозначающий «открытие» замка. Светодиод горит некоторое время, потом гаснет, и система вновь переходит в «режим ввода». Каждый неправильно введённый элемент последовательности должен сопровождаться миганием красного светодиода и сбросом в «начало», каждый правильный – миганием жёлтого. После трёх неправильных вводов начинает мигать красный светодиод, и через некоторое время система вновь возвращается в «режим ввода». Если код не введен до конца за некоторое ограниченное время, происходит сброс в «начало».

Должно быть предусмотрено изменение отпирающей последовательности, что производится следующей последовательностью действий:

– ввод символа «+»;

– ввод новой последовательности, который завершается либо по нажатию *enter*, либо по достижении восьми значений;

– стенд отправляет сообщение произвольного содержания, спрашивая, сделать ли последовательность активной, и запрашивает подтверждение, которое должно быть сделано вводом символа *y*;

– после ввода *y* введённая последовательность устанавливается как активная. Включение/отключение прерываний должно осуществляться нажатием кнопки на стенде и сопровождаться отправкой в последовательный порт сообщения произвольного содержания, сообщающего, какой режим включен (с прерываниями или без прерываний).

screen /dev/cu.usbserial-1101 9600 //команда для терминала

2. Архитектурное описание программы и драйверов UART

Программный комплекс спроектирован на базе модульной неблокирующей архитектуры в рамках кооперативного суперцикла (Super-Loop). Применение блокирующих функций задержки (HAL_Delay) и циклов с активным ожиданием флагов готовности полностью исключено. Синхронизация временных интервалов (таймауты ввода, длительности вспышек и мигания светодиодов) реализована с использованием системного миллисекундного таймера SysTick (функция HAL_GetTick()).

Архитектура программного обеспечения включает следующие структурные модули:

• Модуль кольцевого буфера (ring_buffer.h): легковесный кольцевой FIFO-буфер фиксированного размера (128 байт), обеспечивающий безопасный прием байт в прерывании без гонок данных.
• Модуль драйвера UART (uart_driver.h / uart_driver.c): аппаратно-зависимый уровень управления контроллером USART6. Реализует неблокирующее чтение в режимах опроса и прерываний, отправку данных и обработку прерываний NVIC.
• Модуль кодового замка (lock_fsm.h / lock_fsm.c): Обеспечивает посимвольную обработку потока данных, эхо-вывод, смену пароля и генерацию событий световой индикации.
• Модуль светодиодной индикации (led.h / led.c): неблокирующий драйвер управления зеленым светодиодом (PD13) и двухцветным светодиодом (PD14/PD15).
• Модуль тактовой кнопки (button.h / button.c): цифровой автомат опроса физической кнопки стенда (PC15) с подавлением дребезга контактов (35 мс) для переключения режима прерываний UART.
• Главный модуль (main.c): центральный суперцикл, осуществляющий координацию работы модулей в реальном времени.

3. Аппаратные особенности интерфейса UART на стенде SDK-1.1M

На стенде SDK-1.1M отладочный интерфейс UART подключен ко встроенному аппаратному программатору отладчику на базе специализированного моста FTDI FT2232. Мост преобразует сигналы интерфейса UART в виртуальный COM-порт USB управляющего персонального компьютера.

В соответствии с электрической принципиальной схемой стенда, линии отладочного последовательного порта заведены на периферийный блок USART6 микроконтроллера:

- Вывод PC6 (TX): сконфигурирован в режим альтернативной функции AF8 (USART6_TX) с двухтактным выходным каскадом (Push-Pull). Служит для передачи данных со стенда в терминал ПК.
- Вывод PC7 (RX): сконфигурирован в режим альтернативной функции AF8 (USART6_RX) с внутренней подтяжкой к шине питания (Pull-Up). Служит для приема символов из терминала ПК.

Обозначение	Вывод МК	Конфигурация вывода	Функциональное назначение
USART6_TX	PC6	AF8 (USART6), Push-Pull, High Speed	Передача текстовых данных в консоль ПК
USART6_RX	PC7	AF8 (USART6), Input Pull-Up	Приём командных символов от пользователя
Кнопка SB1	PC15	GPIO Input, Pull-Up	Переключение режима прерываний UART
Зеленый LED	PD13	GPIO Output Push-Pull	Индикация отпирания замка (горит 3 сек)
Желтый LED	PD14	GPIO Output Push-Pull	Одиночная вспышка при верном символе
Красный LED	PD15	GPIO Output Push-Pull	Вспышка при ошибке / тревога блокировки

Таблица 1. Аппаратные выводы стенда SDK-1.1M, задействованные в работе

4. Реализация драйвера UART в режимах опроса и прерываний

Разработанный драйвер uart_driver реализует строгое разделение и динамическое переключение между двумя принципами ввода-вывода данных.

Режим циклического опроса флагов (Polling, без прерываний)

В режиме опроса прерывания контроллера USART6 отключены на аппаратном уровне ядра процессора с помощью вызова функции NVIC_DisableIRQ(USART6_IRQn), а также в регистре управления USART_CR1 путем сброса бита RXNEIE (прерывание по флагу RXNE запрещено).

Неблокирующий прием реализован путем мгновенной проверки флага RXNE в статусном регистре USART_SR: если флаг равен нулю (данные еще не поступили), функция uart_driver_read_char немедленно возвращает false, передавая управление суперциклу. Если флаг равен единице, значение байта считывается из регистра данных USART_DR и возвращается true.

Режим прерываний с кольцевой буферизацией FIFO

При вызове функции uart_driver_set_mode(UART_MODE_INTERRUPT) драйвер переводится в асинхронный режим:

Очищается кольцевой буфер приема s_rx_ring и регистр USART_DR для устранения остаточных байт.
В контрольном регистре USART_CR1 аппаратно взводится бит RXNEIE (разрешение прерывания по приему байта).
В контроллере прерываний NVIC активируется вектор прерывания USART6_IRQn (NVIC_EnableIRQ).

При приёме очередного байта микроконтроллер прерывает основную программу и передает управление в обработчик USART6_IRQHandler. Обработчик считывает байт из регистра данных DR и помещает его в кольцевой буфер через функцию ring_buffer_put. Дополнительно обрабатываются и очищаются флаги аппаратных ошибок приёмника (ORE - переполнение, NE - шумы, FE - ошибка кадра), что исключает зависание приемопередатчика.

5. Конечный автомат кодового замка

Прикладная логика кодового замка структурирована в виде конечного автомата, включающего следующие состояния:

STATE_INPUT_CODE: дежурный режим ожидания и валидации символов пароля. Принимаемые символы зеркалируются обратно в терминал (эхо).
Регистронезависимое сравнение (tolower) сопоставляет ввод с активным паролем (по умолчанию «1234abcd»). При верном символе формируется вспышка желтого светодиода (200 мс). При ошибке генерируется вспышка красного светодиода (300 мс), цепочка ввода сбрасывается в начало, а счетчик ошибок увеличивается. При

отсутствии активности в течение 5 секунд происходит сброс по таймауту. Символ «+» переводит автомат в режим смены пароля.
STATE_UNLOCKED: состояние открытого замка при успешном вводе 8 символов. Загорается зеленый светодиод PD13 на 3 секунды, в терминал выводится «*** ACCESS GRANTED (LOCK OPEN) ***». По истечении 3 секунд замок автоматически запирается.
STATE_BLOCKED_ALARM: аварийная блокировка после 3 неверных попыток подряд. Активируется циклическое мигание красного светодиода PD15 длительностью 5 секунд, выводя предупреждение. Прием символов на время блокировки игнорируется.
STATE_NEW_CODE_INPUT: режим ввода новой последовательности до 8 символов. Ввод завершается нажатием Enter (символы \r или \n) либо при вводе 8 символов.
STATE_NEW_CODE_CONFIRM: интерактивный запрос подтверждения «Make this code active? (y/n): ». При нажатии «y» или «Y» новый пароль вступает в силу. При любом другом символе смена отменяется.

Результаты практического тестирования программы на стенде

Программа была загружена во Flash-память стенда SDK-1.1M. Взаимодействие со стендом производилось через утилиту screen в среде macOS на скорости 9600 бод (/dev/cu.usbserial-1101 9600). В ходе испытаний успешно подтверждены все сценарии:

1. Корректный ввод пароля «1234abcd» приводит к открытию замка и индикации зелёным светодиодом.
2. Незавершенный ввод («1234a») корректно сбрасывается по истечении таймаута.
3. Три ошибочных попытки подряд вызывают срабатывание защитной блокировки.
4. Смена пароля через комбинацию «+» «12345678» «y» успешно обновляет отпирающую комбинацию.
5. Нажатие тактовой кнопки стенда (PC15) безошибочно переключает режим работы драйвера между опросом и прерываниями с отправкой сервисных уведомлений.

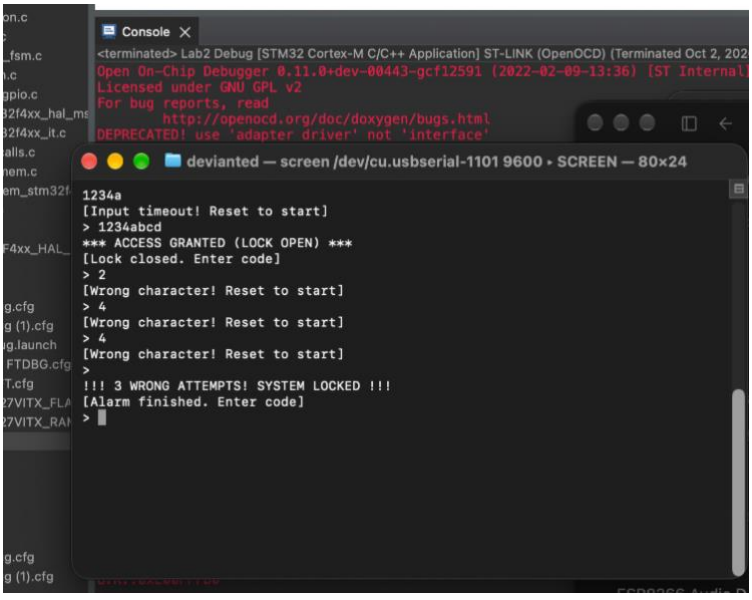


Рисунок 1. Протокол тестирования: таймаут ввода, успешное открытие и срабатывание тревоги

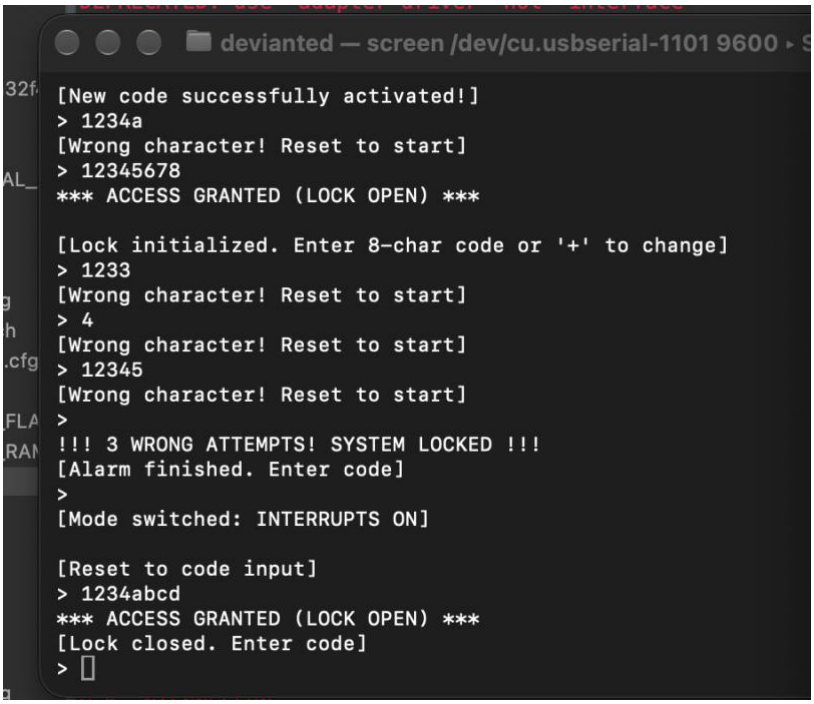


Рисунок 2. Протокол тестирования: процедура смены пароля и переключение режима прерываний

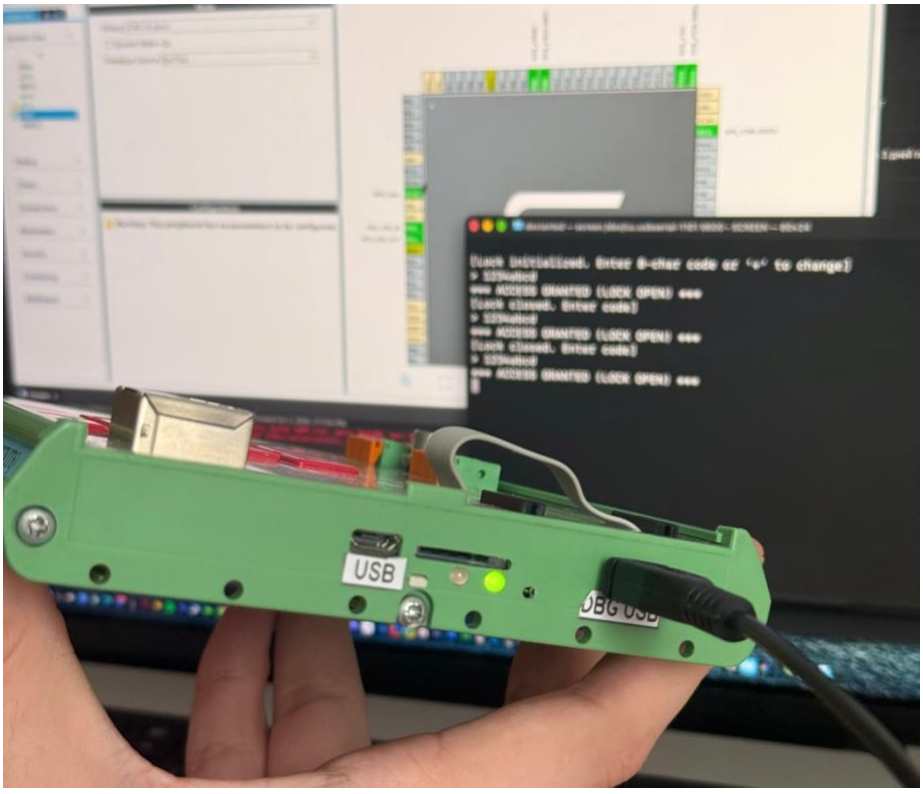


Рисунок 3. Протокол тестирования: процедура смены пароля и переключение режима прерываний

6. Исходные коды программы собственной разработки

Ниже приведены полные исходные коды программных модулей собственной разработки, созданных в рамках выполнения лабораторной работы № 2.

Файл Core/Inc/ring_buffer.h (Кольцевой буфер FIFO)

```
#ifndef INC_RING_BUFFER_H_
#define INC_RING_BUFFER_H_

#include <stdint.h>
#include <stdbool.h>

#define RING_BUFFER_SIZE 128

typedef struct {
    uint8_t buffer[RING_BUFFER_SIZE];
    volatile uint16_t head;
    volatile uint16_t tail;
} RingBuffer;

static inline void ring_buffer_init(RingBuffer *rb) {
    rb->head = 0;
    rb->tail = 0;
}

static inline bool ring_buffer_is_empty(const RingBuffer *rb) {
    return (rb->head == rb->tail);
}

static inline bool ring_buffer_is_full(const RingBuffer *rb) {
    return ((uint16_t)((rb->head + 1) % RING_BUFFER_SIZE) == rb->tail);
}

static inline bool ring_buffer_put(RingBuffer *rb, uint8_t byte) {
    uint16_t next = (uint16_t)((rb->head + 1) % RING_BUFFER_SIZE);
    if (next == rb->tail) {
        return false; /* Буфер переполнен */
    }
    rb->buffer[rb->head] = byte;
    rb->head = next;
    return true;
}

static inline bool ring_buffer_get(RingBuffer *rb, uint8_t *byte) {
    if (rb->head == rb->tail) {
        return false; /* Буфер пуст */
    }
    *byte = rb->buffer[rb->tail];
    rb->tail = (uint16_t)((rb->tail + 1) % RING_BUFFER_SIZE);
    return true;
}

static inline uint16_t ring_buffer_count(const RingBuffer *rb) {
    if (rb->head >= rb->tail) {
        return (rb->head - rb->tail);
    }
    return (RING_BUFFER_SIZE - rb->tail + rb->head);
}

#endif /* INC_RING_BUFFER_H_ */
```

Файл Core/Inc/uart_driver.h (Заголовочный файл драйвера UART)

```
#ifndef INC_UART_DRIVER_H_
#define INC_UART_DRIVER_H_

#include "stm32f4xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

typedef enum {
    UART_MODE_POLLING = 0,   /* Без прерываний (опрос регистров, неблокирующий) */
    UART_MODE_INTERRUPT      /* С использованием прерываний (FIFO буферизация) */
} UartDriverMode;

/* Инициализация драйвера UART */
void uart_driver_init(UART_HandleTypeDef *huart);

/* Переключение режима работы */
void uart_driver_set_mode(UartDriverMode mode);
UartDriverMode uart_driver_get_mode(void);

/* Неблокирующее чтение символа. */
bool uart_driver_read_char(char *out_char);

void uart  driver  write  char(char c);
void uart_driver_write_string(const char *str);

void uart_driver_irq_handler(void);

#endif /* INC_UART_DRIVER_H_ */
```

Файл Core/Src/uart_driver.c (Реализация драйвера UART)

```
#include "uart_driver.h"
#include "ring_buffer.h"

static UART_HandleTypeDef *s_huart = NULL;
static UartDriverMode s_current_mode = UART_MODE_POLLING;

/* Кольцевой буфер приёма для режима прерываний */
static RingBuffer s_rx_ring;

void uart_driver_init(UART_HandleTypeDef *huart)
{
    s_huart = huart;
    ring_buffer_init(&s_rx_ring);

    /* По умолчанию прерывания отключены */
    uart_driver_set_mode(UART_MODE_POLLING);
}

void uart_driver_set_mode(UartDriverMode mode)
{
    if (s_huart == NULL) return;

    s_current_mode = mode;

    if (mode == UART_MODE_INTERRUPT) {
        /* Очищаем буфер перед переходом */
        ring_buffer_init(&s_rx_ring);

        /* Сбрасываем флаг RXNE чтением DR, если там что-то висело */
        __HAL_UART_FLUSH_DRREGISTER(s_huart);
```

```
        /* Включаем прерывание по приёму байта (RXNE) */
        __HAL_UART_ENABLE_IT(s_huart, UART_IT_RXNE);
        NVIC_EnableIRQ(USART6_IRQn);
    } else {
        /* Отключаем прерывание RXNE и глобальное в NVIC */
        __HAL_UART_DISABLE_IT(s_huart, UART_IT_RXNE);
        NVIC_DisableIRQ(USART6_IRQn);
        ring_buffer_init(&s_rx_ring);
    }
}

UartDriverMode uart_driver_get_mode(void)
{
    return s_current_mode;
}

bool uart_driver_read_char(char *out_char)
{
    if (s_huart == NULL || out_char == NULL) {
        return false;
    }

    if (s_current_mode == UART_MODE_INTERRUPT) {
        /* читаем из кольцевого буфера */
        uint8_t byte;
        if (ring_buffer_get(&s_rx_ring, &byte)) {
            *out_char = (char)byte;
            return true;
        }
        return false;
    } else {
        /* проверяем флаг RXNE без зависания */
        if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_RXNE) != RESET) {
            /* Байт поступил — считываем его из регистра данных DR */
            *out_char = (char)(s_huart->Instance->DR & 0xFF);
            return true;
        }
        return false;
    }
}

void uart_driver_write_char(char c)
{
    if (s_huart == NULL) return;

    /* Ждем готовности передатчика (TXE) */
    while (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_TXE) == RESET) {
    }
    s_huart->Instance->DR = (uint16_t)c;
}

void uart_driver_write_string(const char *str)
{
    if (str == NULL) return;
    while (*str) {
        uart_driver_write_char(*str++);
    }
}

void uart_driver_irq_handler(void)
{
    if (s_huart == NULL) return;

    /* Проверяем флаг поступления байта RXNE */
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_RXNE) != RESET &&
        __HAL_UART_GET_IT_SOURCE(s_huart, UART_IT_RXNE) != RESET)
    {
        uint8_t received_byte = (uint8_t)(s_huart->Instance->DR & 0xFF);
```

```
        /* Сохраняем в кольцевой буфер */
        ring_buffer_put(&s_rx_ring, received_byte);
    }

    /* Обработка ошибок (Overrun, Noise, Framing), чтобы порт не блокировался */
    if (   HAL_UART_GET_FLAG(s_huart, UART_FLAG_ORE) != RESET) {
        __HAL_UART_CLEAR_OREFLAG(s_huart);
    }
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_NE) != RESET) {
        __HAL_UART_CLEAR_NEFLAG(s_huart);
    }
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_FE) != RESET) {
        __HAL_UART_CLEAR_FEFLAG(s_huart);
    }
}
```

Файл Core/Src/lock_fsm.c (Реализация конечного автомата замка)

```
#include "lock_fsm.h"
#include "led.h"
#include "uart_driver.h"
#include <ctype.h>
#include <string.h>
#include <stdio.h>

#define CODE_MAX_LEN        8U
#define INPUT_TIMEOUT_MS    5000U
#define UNLOCKED_TIME_MS    3000U
#define ALARM_TIME_MS       5000U
#define FLASH_YELLOW_MS     200U
#define FLASH_RED_MS        300U

typedef enum {
    STATE_INPUT_CODE = 0,
    STATE_UNLOCKED,
    STATE_BLOCKED_ALARM,
    STATE_NEW_CODE_INPUT,
    STATE_NEW_CODE_CONFIRM
} LockState;

static LockState s_state = STATE_INPUT_CODE;

/* Активный пароль (по умолчанию "1234abcd") */
static char s_active_code[CODE_MAX_LEN + 1] = "1234abcd";

/* Буфер для ввода нового пароля при смене */
static char s_temp_code[CODE_MAX_LEN + 1];
static uint8_t s_temp_code_len = 0;
static uint8_t s_input_idx = 0;
static uint8_t s_error_count = 0;
static uint32_t s_last_input_tick = 0;
static uint32_t s_state_end_tick = 0;

static bool char_match(char a, char b)
{
    return (tolower((unsigned char)a) == tolower((unsigned char)b));
}

void lock_fsm_init(void)
{
    s_state = STATE_INPUT_CODE;
    s_input_idx = 0;
    s_error_count = 0;
    s_last_input_tick = 0;
    s_state_end_tick = 0;
    led_set(LED_ALL, LED_STATE_OFF);
```

```
    uart_driver_write_string("\r\n[Lock initialized. Enter 8-char code or '+' to change]\r\n> ");
}

void lock_fsm_reset(void)
{
    s_state = STATE_INPUT_CODE;
    s_input_idx = 0;
    s_error_count = 0;
    s_last_input_tick = 0;
    led_stop_effects();
    led_set(LED_ALL, LED_STATE_OFF);
    uart_driver_write_string("\r\n[Reset to code input]\r\n> ");
}

void lock_fsm_process_char(char c)
{
    /* Игнорируем ввод, если замок сейчас открыт или заблокирован */
    if (s_state == STATE_UNLOCKED || s_state == STATE_BLOCKED_ALARM) {
        return;
    }

    /* Эхо-вывод принятого символа в консоль */
    uart_driver_write_char(c);

    uint32_t now = HAL_GetTick();

    switch (s_state) {
        case STATE_INPUT_CODE:
            /* переход в режим смены пароля по символу + */
            if (c == '+') {
                s_state = STATE_NEW_CODE_INPUT;
                s_temp_code_len = 0;
                memset(s_temp_code, 0, sizeof(s_temp_code));
                uart_driver_write_string("\r\n[Enter new code (up to 8 chars, press Enter to finish)]:\r\n> ");
                return;
            }

            /* Игнорируем пробелы переносы строк */
            if (c == '\r' || c == '\n') {
                return;
            }

            s_last_input_tick = now;

            /* Проверяем соответствие текущему символу активного пароля */
            if (char_match(c, s_active_code[s_input_idx])) {
                s_input_idx++;
                led_trigger_flash(LED_YELLOW, FLASH_YELLOW_MS);

                if (s_input_idx >= CODE_MAX_LEN) {
                    /* Замок открыт */
                    s_input_idx = 0;
                    s_error_count = 0;
                    led_stop_effects();
                    led_set(LED_GREEN, LED_STATE_ON);
                    s_state = STATE_UNLOCKED;
                    s_state_end_tick = now + UNLOCKED_TIME_MS;
                    uart_driver_write_string("\r\n*** ACCESS GRANTED (LOCK OPEN) ***\r\n");
                }
            } else {
                s_input_idx = 0;
                s_error_count++;
                /* сброс в начало */
                led_trigger_flash(LED_RED, FLASH_RED_MS);
                uart_driver_write_string("\r\n[Wrong character! Reset to start]\r\n> ");

                if (s_error_count >= 3) {
                    /* после 3 ошибок система блокируется */
                    s_error_count = 0;
```

```
                s_state = STATE_BLOCKED_ALARM;
                s_state_end_tick = now + ALARM_TIME_MS;
                led_start_blink(LED_RED, 200, ALARM_TIME_MS);
                uart_driver_write_string("\r\n!!! 3 WRONG ATTEMPTS! SYSTEM LOCKED !!!\r\n");
            }
        }
        break;

    case STATE_NEW_CODE_INPUT:
        /* Окончание ввода либо Enter , либо 8 символов */
        if (c == '\r' || c == '\n') {
            if (s_temp_code_len > 0) {
                s_temp_code[s_temp_code_len] = '\0';
                s_state = STATE_NEW_CODE_CONFIRM;
                uart_driver_write_string("\r\nMake this code active? (y/n): ");
            } else {
                /* Пустой пароль нельзя ;( */
                uart_driver_write_string("\r\n[Empty code. Cancelled]\r\n> ");
                s_state = STATE_INPUT_CODE;
            }
        } else {
            if (s_temp_code_len < CODE_MAX_LEN) {
                s_temp_code[s_temp_code_len++] = c;
                if (s_temp_code_len == CODE_MAX_LEN) {
                    s_temp_code[s_temp_code_len] = '\0';
                    s_state = STATE_NEW_CODE_CONFIRM;
                    uart_driver_write_string("\r\nMake this code active? (y/n): ");
                }
            }
        }
        break;

    case STATE_NEW_CODE_CONFIRM:
        if (c == 'y' || c == 'Y') {
            /* новый пароль становится активным */
            strncpy(s_active_code, s_temp_code, CODE_MAX_LEN + 1);
            s_state = STATE_INPUT_CODE;
            s_input_idx = 0;
            s_error_count = 0;
            uart_driver_write_string("\r\n[New code successfully activated!]\r\n> ");
        } else {
            s_state = STATE_INPUT_CODE;
            s_input_idx = 0;
            uart_driver_write_string("\r\n[Code change cancelled]\r\n> ");
        }
        break;

    default:
        break;
  }
}

void lock_fsm_update(uint32_t current_tick)
{
  switch (s_state) {
    case STATE_INPUT_CODE:
        /* если ввели не до конца за 5 сек - сброс */
        if (s_input_idx > 0) {
            if ((current_tick - s_last_input_tick) >= INPUT_TIMEOUT_MS) {
                s_input_idx = 0;
                led_trigger_flash(LED_RED, FLASH_RED_MS);
                uart_driver_write_string("\r\n[Input timeout! Reset to start]\r\n> ");
            }
        }
        break;

    case STATE_UNLOCKED:
        /* Зелёный горит некоторое время, затем возвращаемся в режим ввода */
        if ((int32_t)(current_tick - s_state_end_tick) >= 0) {
```

```
            led_set(LED_GREEN, LED_STATE_OFF);
            s_state = STATE_INPUT_CODE;
            s_input_idx = 0;
            uart_driver_write_string("[Lock closed. Enter code]\r\n> ");
        }
        break;

    case STATE_BLOCKED_ALARM:
        /* блокировка завершилась */
        if ((int32_t)(current_tick - s_state_end_tick) >= 0) {
            led_stop_effects();
            led_set(LED_RED, LED_STATE_OFF);
            s_state = STATE_INPUT_CODE;
            s_input_idx = 0;
            uart_driver_write_string("[Alarm finished. Enter code]\r\n> ");
        }
        break;

    default:
        break;
  }
}
```

Файл Core/Src/main.c (Фрагмент главного суперцикла программы)

```
int main(void)
{
 /* USER CODE BEGIN 1 */

 /* USER CODE END 1 */

 /* MCU Configuration--------------------------------------------------------*/

 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
 HAL_Init();

 /* USER CODE BEGIN Init */

 /* USER CODE END Init */

 /* Configure the system clock */
 SystemClock_Config();

 /* USER CODE BEGIN SysInit */

 /* USER CODE END SysInit */

 /* Initialize all configured peripherals */
 MX_GPIO_Init();
 MX_USART6_UART_Init();
 /* USER CODE BEGIN 2 */
 led_init();
 button_init();
 uart_driver_init(&huart6);
 lock_fsm_init();
 /* USER CODE END 2 */

 /* Infinite loop */
 /* USER CODE BEGIN WHILE */
 while (1)
 {
  uint32_t now = HAL_GetTick();

  /* опрос кнопки стенда для переключения режима прерываний */
```

```
  ButtonEvent btn_ev = button_update(now);
  if (btn_ev != BTN_EVENT_NONE) {
    /* По нажатию кнопки переключаем режим: polling <-> interrupts */
    if (uart_driver_get_mode() == UART_MODE_POLLING) {
      uart_driver_set_mode(UART_MODE_INTERRUPT);
      uart_driver_write_string("\r\n[Mode switched: INTERRUPTS ON]\r\n");
    } else {
      uart_driver_set_mode(UART_MODE_POLLING);
      uart_driver_write_string("\r\n[Mode switched: INTERRUPTS OFF (Polling)]\r\n");
    }
    /* при смене режима сбрасывать ввод замка */
    lock_fsm_reset();
  }

  /* символа из UART */
  char rx_char;
  if (uart_driver_read_char(&rx_char)) {
    lock_fsm_process_char(rx_char);
  }

  /* обновление конечного автомата замка и анимаций светодиодов */
  lock_fsm_update(now);
  led_update(now);

  /* USER CODE END WHILE */
```

Файл Core/Src/stm32f4xx_it.c (Обработчик прерывания USART6_IRQHandler)

```
/* USER CODE END 0 */

/**
  * @brief This function handles USART6 global interrupt.
  */
void USART6_IRQHandler(void)
{
  /* USER CODE BEGIN USART6_IRQn 0 */
  uart_driver_irq_handler();
  /* USER CODE END USART6_IRQn 0 */
  /* USER CODE BEGIN USART6_IRQn 1 */

  /* USER CODE END USART6_IRQn 1 */
}

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */
```

7. Выводы

В ходе выполнения работы мы освоили принципы работы с последовательным интерфейсом UART и системой аппаратных прерываний NVIC на микроконтроллере STM32.

Научились организовывать обмен данными с компьютером в двух режимах — как по циклическому опросу флагов, так и по прерываниям с использованием собственного кольцевого буфера (FIFO), чтобы данные не терялись при интенсивном потоке. Разобрались, как строить неблокирующий ввод-вывод без использования зависающих задержек, и перевели логику кодового замка на управление через терминал с поддержкой эхо-печати,

таймаутов, защиты от перебора и смены пароля на лету. Также реализовали оперативное переключение между прерываниями и опросом по нажатию кнопки на стенде. Программу успешно загрузили на стенд SDK-1.1M, протестировали через терминал и убедились в её корректной и стабильной работе во всех предусмотренных сценариях.